Flora-v2 코드 리뷰

1. 프로젝트 개요

- 한 줄 설명: FLORA v2(문서 버전 3.0, 2026-06-15). 1인 창작자가 AI로 전자책·사진집·룩북·오디오북을 기획부터 PDF/ePUB/음성까지 만드는 멀티테넌트 출판 SaaS를 v1(flora)에서 구조를 다시 잡아 재작성한 버전 (CLAUDE.md, PRD.md).
- 목적/대상 사용자: 원고는 있지만 편집·출판 경험이 없는 1인 작가, 에세이스트, 인플루언서, 소규모 독립 출판사 (PRD.md 1.4).
- 저장소 상태: platform 저장소의 untracked 폴더. 자체 .git 없음. api/ 아래에 가상환경이 없고(CLAUDE.md가 언급한 .venv311, f5-tts/ 디렉토리 모두 부재), 프론트만 node_modules가 설치되어 있다.

기술 스택

레이어	기술	근거
프론트	Next.js 16.2.6 (App Router, src/proxy.ts), React 19.2, TypeScript, Tailwind v4, Shadcn v4(@base-ui), @xyflow/react, react-dropzone, canvas-confetti	frontend/package.json
백엔드	FastAPI 0.115, Uvicorn, Pydantic v2, python-jose(HS256), supabase-py 2.10 (동기 클라이언트)	api/requirements.txt, api/dependencies.py
DB/인증	Supabase PostgreSQL + Auth, RLS, 마이그레이션 3개	supabase/migrations/001~003
AI(텍스트)	Anthropic AsyncAnthropic 단일 제공자(claude-sonnet-4-6), OpenRouter 키는 설정만 존재	api/services/llm.py:12-24
AI(이미지)	없음. placehold.co 플레이스홀더 URL 반환	api/routers/cover.py:32-41
TTS/STT	F5-TTS 외부 서버 프록시(결과 폐기), OpenAI Whisper STT	api/routers/tts.py:34-85
PDF/ePUB	fpdf2(Helvetica) 폴백만 실제 동작, 수제 ePUB2 zip. WeasyPrint는 requirements에만 있음	api/routers/export.py
결제	Stripe Checkout(가격 ID 플레이스홀더), Webhook(시크릿 오용)	api/routers/billing.py
인프라	배포 설정 없음 (Dockerfile, railway/render, CI 모두 부재)	디렉토리 전체

현재 완성도 추정: 약 30%

- 근거(완료): 기획 문서(PRD 547줄, DESIGN 689줄, CLAUDE 516줄)가 상세하고, 프론트가 10단계 위저드를 단계별 컴포넌트(components/ebook/*, components/project/*)로 분해했다. 60인 페르소나 데이터(lib/personas.ts 724줄), 6종 판형 Auto-Fit 프리셋, 7종 테마, 플랜 한도 표(lib/plan/plan-limits.ts)가 갖춰져 있다. LLM 호출은 비동기 SDK로 바뀌었고 Supabase 마이그레이션에 RLS가 들어갔다.
- 근거(미완): 프론트-백엔드 계약이 여러 곳에서 깨져 있어 엔드투엔드로 완주되는 흐름이 없다. 로그인 후 세션 쿠키가 생기지 않고, 어떤 fetch도 Bearer 토큰을 보내지 않으며, 업로드·전체 집필·내보내기·오디오북·표지가 모두 계약 불일치나 플레이스홀더다. tsc --noEmit이 실패해 next build도 통과하지 못한다. v1의 검증된 엔진(다중 LLM, 7종 표지, WeasyPrint/Typst, F5 래퍼)이 이식되지 않았다.

핵심 기능 목록 (설계 기준)

- 전자책 10단계: 판형 → 장르 → 페르소나(60종) → AI 제목 → 기획안 3종 → 목차 3종 → AI 집필(섹션/전체 SSE/이어쓰기·살붙이기·다듬기) → 표지 → 메타데이터 → PDF/ePUB 내보내기
- 원고 업로드 경로(txt/md/docx/pdf/hwp 파싱 → 기획 검토 → 완성), 나만의 페르소나 합성(STT 포함)

- 오디오북(7종 보이스), 자동화 3종(React Flow 캔버스, Make Webhook, n8n), 4단계 플랜(free/basic/pro/max), 관리자 페이지, 대기자 명단

2. 구조 분석

경로	역할	비고
api/main.py (48줄)	FastAPI 앱, CORS, 11개 라우터 등록	/api/auth/는 아무 검증 없이 authenticated 반환
api/dependencies.py (34줄)	get_current_user HS256 JWT 검증	일부 라우터만 사용
api/supabase_client.py (4줄)	service role 동기 클라이언트 싱글턴	URL 미설정 시 None
api/services/llm.py (67줄)	chat/stream_chat, JSON 파서, 14종 Taste 프롬프트	프론트 60종과 불일치
api/routers/llm.py (344줄)	제목·기획안·페르소나 합성·목차·섹션·전체(SSE)·보조·메타데이터·번역	11개 엔드포인트 전부 무인증
api/routers/upload.py (186줄)	업로드 에코, txt/md/docx/pdf/hwp 파서	프론트와 계약 불일치
api/routers/export.py (173줄)	PDF(fpdf2), ePUB 조립	한글 미지원, 무인증
api/routers/projects.py, settings.py, automation.py, billing.py, tts.py, cover.py, books.py, waitlist.py	CRUD, 설정, Make/n8n, Stripe, TTS, 표지, 병합, 대기자	다수가 스텁
supabase/migrations/001~003	projects, user_settings, user_plans, automation_workflows, waitlist, RLS	프로필/플랜 자동 생성 트리거 없음
frontend/src/proxy.ts	세션 확인 후 /dashboard, /admin 보호	쿠키 세션 전제
frontend/src/app/(dashboard)/dashboard/projects/new/ebook/page.tsx (154줄)	10단계 위저드 셸	v1 4,095줄에서 대폭 축소
frontend/src/app/(dashboard)/dashboard/projects/new/upload/page.tsx (491줄)	업로드 경로 위저드 + 완성 화면	가장 큰 페이지
frontend/src/hooks/use-ebook-step.ts, use-upload-step.ts, use-write-all.ts	위저드 상태(localStorage), 전체 집필 SSE	DB 저장 없음
frontend/src/lib/personas.ts, auto-fit/, theme/, plan/, taste/, skill/	도메인 데이터, 프리셋	taste/, skill/은 미사용

데이터 흐름 요약

브라우저의 클라이언트 컴포넌트가 NEXT_PUBLIC_API_URL(FastAPI)로 직접 fetch하고, 위저드 상태는 전부 localStorage(flora_ebook_draft_new)에만 저장된다. 의도된 설계는 "Supabase 세션 → Bearer 토큰 → FastAPI가 JWT 검증 → service role로 DB 접근"이지만, 프론트에는 Supabase 세션을 만드는 코드도 토큰을 붙이는 코드도 없다(grep Bearer 결과 0건). LLM 엔드포인트는 무인증이라 호출은 되지만 결과는 DB에 저장되지 않고, 인증이 필요한 projects/settings/tts/cover 호출은 모두 401이 된다. 최종 내보내기는 원고 본문을 보내지 않으므로 빈 책이 생성된다.

3. 발견된 문제점

#	심각도	분류	위치(경로:줄)	내용
1	높음	버그	frontend/src/app/(auth)/login/page.tsx:27-45, frontend/src/proxy.ts:26-38	로그인이 GoTrue REST를 raw fetch로 호출하고 받은 토큰을 버린 채 /dashboard로 이동. 세션 쿠키가 없으므로 proxy가 다시 /login으로 돌려보냄

#	심각도	분류	위치(경로:줄)	내용
2	높음	버그	frontend/src 전체 (Bearer 0건), api/routers/projects.py:30-84, settings.py:18-31, tts.py:35, cover.py:33	프론트가 credentials: 'include'만 쓰고 Authorization 헤더를 보내지 않음. 백엔드는 HTTPBearer만 읽으므로 인증 필요 API 전부 401
3	높음	보안	api/routers/llm.py:25,60,102,152,197,223,263,275,287, 305,336, export.py:23,45, upload.py:13,22, books.py:13,27,72	LLM 11개, 내보내기, 업로드, 병합 엔드포인트 무인증. 누구나 서버 Anthropic 키로 무제한 호출(v1 #1 미해결)
4	높음	보안	supabase/migrations/002_rls.sql:13	user_plans에 FOR ALL USING (auth.uid() = user_id) → 로그인 사용자가 anon 키로 자기 플랜을 max로 UPDATE 가능(v1 #3 미해결)
5	높음	보안	frontend/src/components/ui/password-unlock-modal .tsx:14-15, frontend/src/hooks/use-is-pro.ts:14-17	비밀번호 '1234'가 번들에 포함, localStorage 값만으로 max 플랜. 백엔드 플랜 강제 없음(v1 #6 미해결)
6	높음	버그	frontend/src/hooks/use-manuscript-upload.ts:31-44, new/upload/page.tsx:37-49, components/ebook/my-persona-input.tsx:110-121 vs api/routers/upload.py:13-23	프론트는 file_id를 받아 JSON으로 /parse 호출, 백엔드는 file_id를 반환하지 않고 /parse는 multipart file을 요구 → 422. 동일 로직 3곳 복제
7	높음	버그	frontend/src/hooks/use-write-all.ts:28, api/routers/llm.py:223	전체 순차 집필이 EventSource(GET, 바디 없음)로 호출, 백엔드는 POST + TOC 바디 → 405. 7단계 핵심 기능 불능
8	높음	버그	frontend/src/components/ebook/completion-screen.t sx:33-36,55-58, api/routers/export.py:15-20,95-111	내보내기 요청에 content_html/title이 없어 빈 책 생성. fpdf2 Helvetica는 한글을 인코딩하지 못해 예외 → 500. 본문도 3,000자에서 절단
9	높음	설정	frontend/src/app/(dashboard)/dashboard/projects/ne w/ebook/page.tsx:99,108	tsc --noEmit 실패(TocNode, ManuscriptSection 미import) → next build 실패로 배포 불가
10	중간	보안	api/config.py:6, api/dependencies.py:17-22	SUPABASE_JWT_SECRET 기본값이 빈 문자열이고 기동 시 검증 없음. 누락 시 빈 키로 서명한 위조 토큰이 통과할 수 있음. verify_aud=False
11	중간	보안	frontend/src/app/auth/callback/route.ts:7,13	next 파라미터를 검증 없이 ${origin}${next}로 연결 → next=@evil.com 형태로 외부 리다이렉트 가능
12	중간	보안	api/routers/billing.py:64, billing.py:10-14,17-19	웹훅 서명 검증에 stripe_secret_key를 사용(웹훅 시크릿 필요) → 모든 웹훅 400. 이벤트 처리 로직 없음, 가격 ID 플레이스홀더, /plan은 항상 free 하드코딩
13	중간	보안	api/routers/export.py:75,88-90,138,158, books.py:58-60	제목·저자·본문을 escape 없이 HTML/XML에 삽입. ePUB XML이 &, < 한 글자로 깨지고, HTML 미리보기에 스크립트 주입 가능
14	중간	성능	api/routers/projects.py:31,49,57, settings.py:20, automation.py:19,48, billing.py:43, upload.py:108,115-126	async def 안에서 동기 supabase-py, 동기 Stripe, pdfplumber/mammoth 파싱 호출 → 이벤트 루프 블로킹
15	중간	성능	api/routers/upload.py:18-19, tts.py:67,77	업로드 크기 제한 없음. /manuscript는 파일 전체를 hex 문자열(2배)로 되돌려 보냄. STT·ref 업로드도 무제한 메모리 적재
16	중간	버그	api/routers/upload.py:129-140	HWP BodyText/Section0은 zlib 압축 레코드 구조인데 UTF-16 디코드만 수행 → 깨진 문자열
17	중간	버그	api/services/llm.py:46-67 vs frontend/src/lib/personas.ts	프론트 페르소나 60종 중 14종만 백엔드 Taste 프롬프트 보유. 나머지 46종 선택 시 문체 지시가 빈 문자열

#	심각도	분류	위치(경로:줄)	내용
18	중간	버그	api/routers/llm.py:305-326	메타데이터 생성이 프로젝트 ID 문자열만 프롬프트에 넣고 원고·제목을 조회하지 않음 → 내용과 무관한 메타데이터 환각
19	중간	버그	frontend/src/app/(dashboard)/dashboard/projects/new/ebook/page.tsx:18, hooks/use-ebook-step.ts:81	모든 신규 전자책이 projectId='new' 동일 localStorage 키를 공유. DB 프로젝트를 생성하지 않아 "나의 프로젝트"에 남지 않음
20	중간	버그	api/routers/tts.py:44,65-68, frontend/.../audiobook/page.tsx:39-50	TTS 결과를 버리고 존재하지 않는 /api/tts/audio/{id} URL만 반환. 프론트는 JSON 응답을 audiobook.mp3로 저장. ref 업로드도 파일을 저장하지 않음
21	중간	설정	api/requirements.txt, api/routers/waitlist.py:6	EmailStr 사용하나 email-validator 미기재(전이 의존이 없으면 앱 import 실패). weasyprint, numpy, soundfile, markdown은 미사용
22	중간	설정	.env, .env.local (루트, 동일 내용)	루트 env에 SUPABASE_SERVICE_ROLE_KEY, OPENROUTER_API_KEY 실값 존재(값은 확인하지 않음, 키 이름만 확인). 어떤 코드도 루트 env를 읽지 않으며 키 확산 경로
23	낮음	버그	api/routers/projects.py:57, settings.py:20, automation.py:48	.single()은 0행일 때 예외 → 404 대신 500. 신규 사용자의 설정 조회가 항상 500(v1 #16 미해결)
24	낮음	버그	api/routers/automation.py:35, projects.py:80	실패 시 HTTPException을 raise하지 않고 return. update_status는 임의 문자열을 상태로 저장
25	낮음	버그	frontend/src/components/ebook/completion-screen.tsx:75, hooks/use-is-pro.ts:20,27	/share/{id} 라우트 없음(404). flora_user_email은 어디서도 저장하지 않음, /api/billing/plan을 Next 상대경로로 호출(라우트 없음)
26	낮음	품질	api/routers/cover.py:32-41, books.py:72-77, automation.py:46-53, api/main.py:46-48	표지·쪽번호 삽입·Make 트리거·/api/auth/가 기능 없는 스텁인데 성공 응답 반환
27	낮음	품질	frontend/src/lib/skill/skill-runner.ts, lib/taste/taste-injector.ts, components/project/manuscript-uploader.tsx	어디서도 import되지 않는 죽은 코드
28	낮음	품질	CLAUDE.md:22-47, frontend/CLAUDE.md	문서가 flora/, f5-tts/, .venv311 등 v2에 없는 구조를 설명. 환경 문서와 실제 불일치
29	낮음	품질	supabase/migrations/002_rls.sql:22-24, frontend/src/lib/admin.ts:1	관리자 이메일 하드코딩(DB 정책과 프론트 양쪽). 관리자 페이지 통계는 0 고정

심각도 높음 항목 상세

#1 로그인 세션 미생성. 로그인 페이지는 @supabase/ssr의 브라우저 클라이언트를 쓰지 않고 /auth/v1/token을 직접 호출한 뒤 응답 토큰을 저장하지 않는다. proxy.ts는 쿠키 기반 getUser()로 보호 경로를 판정하므로 사용자는 /dashboard에 도달하지 못하고 /login으로 되돌아간다. createClient().auth.signInWithPassword() / signUp({ options: { emailRedirectTo } })로 교체하면 쿠키가 생기고 콜백 라우트도 정상 경로가 된다.

#2 토큰 미전송. 백엔드는 HTTPBearer만 읽는데 프론트의 모든 fetch는 Bearer 헤더 없이 크로스 오리진 쿠키만 보낸다(그 쿠키도 FastAPI가 읽지 않는다). 결과적으로 프로젝트 목록·상세·시드 복제·설정 저장·TTS·표지가 전부 401이다. lib/api.ts에 세션의 access_token을 주입하는 단일 apiFetch를 만들고 모든 raw fetch를 교체해야 한다.

#3 LLM·내보내기 무인증. v1과 동일한 비용 유출 구조다. CORS는 브라우저만 막고 curl은 막지 못한다. /write-all은 TOC 길이에 제한이 없어 한 요청으로 수백 번의 Claude 호출을 유발할 수 있다. 라우터 단위 dependencies=[Depends(get_current_user)]와 TOC 길이·본문 길이 상한, 플랜별 크레딧 차감이 필요하다.

#4 user_plans RLS. FOR ALL 정책은 본인 행에 대한 UPDATE/INSERT를 허용한다. 프론트가 anon 키 클라이언트를 갖고 있으므로 브라우저 콘솔에서 supabase.from('user_plans').update({plan:'max',

credits_remaining:-1})가 가능하다. SELECT 전용 정책으로 바꾸고 쓰기는 service role 백엔드만 하도록 마이그레이션 004가 필요하다.

#5 클라이언트 전용 Pro 게이팅. localStorage.flora_pro_unlocked='true' 한 줄로 max 플랜 UI가 열리며, 비밀번호는 번들에 평문으로 있다. 백엔드는 플랜을 전혀 확인하지 않는다(billing.py:17-19는 항상 free). 수익 모델이 코드로 보호되지 않는다.

#6 업로드 계약 불일치. 업로드 경로 위저드, 원고 업로더 훅, 나만의 페르소나 샘플 업로드 3곳이 모두 존재하지 않는 file_id 기반 2단계 프로토콜을 쓴다. 백엔드 /parse가 multipart 파일을 직접 받으므로 프론트를 단일 multipart 호출로 바꾸고 훅 하나로 합치는 것이 가장 작은 수정이다.

#7 전체 집필 SSE 불능. EventSource는 GET만 지원하고 바디를 보낼 수 없어 TOC가 전달되지 않는다. fetch(POST) + response.body.getReader()로 SSE 프레임을 파싱하도록 바꾸고, 결과를 manuscript 상태에 반영해야 한다.

#8 내보내기 결과물 없음. 완성 화면은 project_id와 watermark만 보내고, 백엔드는 요청 바디의 content_html만 사용한다. 또 settings.weasyprint_server_url이 Settings에 정의되지 않아(export.py:28의 hasattr가 항상 False) 항상 fpdf2 폴백으로 가며, 기본 Helvetica 폰트는 한글을 인코딩하지 못해 예외가 난다. v1의 WeasyPrint + Noto CJK 경로를 이식하고, 서버가 DB에서 원고를 조회해 HTML을 만들도록 해야 한다.

#9 빌드 실패. new/ebook/page.tsx가 TocNode, ManuscriptSection 타입을 import하지 않아 타입 검사가 실패한다. Next.js는 빌드 중 타입 오류로 중단하므로 Vercel 배포가 불가능하다. 수정은 import 한 줄이지만, CI가 없어 이런 회귀를 잡지 못하는 것이 근본 문제다.

4. 효율성 개선 제안

구조/아키텍처

- 인증 경계를 한 곳으로: 백엔드는 APIRouter(dependencies=[Depends(get_current_user)])로 라우터 단위 적용, 공개 엔드포인트(/health, /api/waitlist, /api/billing/webhook)만 예외로 둔다. 프론트는 lib/api.ts의 apiFetch(토큰 주입, 401 시 로그인 이동, JSON/에러 타입)로 raw fetch 25곳을 교체한다.
- 원고는 DB가 단일 소스: 위저드 시작 시 POST /api/projects로 ID를 받고 ebook_step_state를 debounce PATCH한다. 내보내기·메타데이터·TTS는 project_id만 받아 서버가 DB에서 원고를 읽도록 해 클라이언트가 큰 HTML을 보내지 않게 한다.
- 페르소나 단일 소스: lib/personas.ts를 JSON으로 분리해 백엔드가 같은 파일을 읽거나, 프론트가 systemPromptSnippet을 요청에 포함하고 백엔드는 길이 제한만 둔다.
- v1 엔진 이식 계층: api/services/에 llm_router.py(v1 call_llm 비동기화), cover_providers.py, pdf_renderer.py(WeasyPrint/Typst), tts_client.py를 두고 라우터는 얇게 유지한다.

성능

- supabase-py 동기 클라이언트는 run_in_threadpool로 감싸거나 acreate_client(AsyncClient)로 교체. Stripe 호출도 asyncio.to_thread.
- 파일 파싱(pdfplumber, mammoth)과 PDF 렌더는 스레드풀로 격리하고, 업로드는 20MB 상한과 SpooledTemporaryFile 스트리밍 처리. /manuscript의 hex 에코는 제거.
- 전체 집필은 섹션별 순차 호출(llm.py:228-250)을 세마포어 2~3의 제한 병렬로 바꾸고, 결과를 서버에서 바로 DB에 저장해 연결이 끊겨도 진행분이 남게 한다.
- TTS 산출물은 Supabase Storage 업로드 후 서명 URL 반환.

개발 생산성 (빌드, 타입, 테스트, 중복 제거)

- CI 신설: npm run lint && npx tsc --noEmit && npm run build, pytest(401·업로드 한도·웹훅 서명). 현재 존재하는 타입 오류 2건이 바로 잡혔을 것이다.
- 업로드 로직 3중 복제를 use-manuscript-upload 하나로, 내보내기 로직 3중 복제(completion-screen.tsx, upload/page.tsx:363-400, projects/[id]/page.tsx:34-55)를 useExport 훅 하나로 통합.

- requirements.txt를 실제 import 기준으로 정리하고 pyproject.toml + lock(uv/pip-tools)으로 전환. Python 버전 3.11 고정.
- 죽은 코드(skill-runner, taste-injector, manuscript-uploader) 삭제 또는 실제 연결.

운영/배포

- 배포 설정이 전무하다. api/Dockerfile(python:3.11-slim + libpango-1.0-0 libpangoft2-1.0-0 fonts-noto-cjk), Railway/Render 설정, Vercel 프로젝트 env 체크리스트를 만든다.
- 필수 env 기동 검증: SUPABASE_URL, SUPABASE_JWT_SECRET(또는 JWKS URL), ANTHROPIC_API_KEY, STRIPE_WEBHOOK_SECRET이 비면 앱이 시작하지 않도록 Settings에 validator 추가.
- 루트 .env/.env.local(동일 파일) 삭제, 서비스 롤 키와 OpenRouter 키는 api/.env에만 두고 노출 이력이 있으면 키 재발급.
- CLAUDE.md를 실제 v2 구조로 갱신(F5-TTS는 외부 서버, venv 경로, 실행 명령).

5. 수정 작업 목록 및 권장 모델

#	작업	우선순위	예상 규모	권장 모델	선정 이유
1	인증 체계 복구: 로그인/회원가입을 createBrowserClient().auth.signInWithPassword/signUp으로 교체, 로그아웃 추가, auth/callback의 next를 /로 시작하고 //·@가 없는 경로만 허용, lib/api.ts apiFetch(access_token 주입), 백엔드 llm/export/upload/books/automation 라우터에 Depends(get_current_user) 일괄 적용, JWT secret 미설정 시 기동 실패 처리	P0	L	Fable 5.1	인증 로직이 프론트·백엔드·DB 정책에 걸친 횡단 변경, 실수 시 비용 유출
2	플랜/결제 서버 강제: 마이그레이션 004(user_plans SELECT 전용 정책, 가입 시 user_plans/user_settings 생성 트리거), STRIPE_WEBHOOK_SECRET 분리와 checkout.session.completed/customer.subscription.* 처리, /billing/plan DB 조회, LLM 호출 시 크레딧 차감, PasswordUnlockModal과 localStorage 우회 제거	P0	M	Fable 5.1	결제·권한 상승 차단, 데이터 모델 정책 변경
3	빌드 복구: new/ebook/page.tsx에 TocNode, ManuscriptSection import 추가, npx tsc --noEmit·npm run lint·npm run build 통과 확인, 미사용 skill-runner.ts·taste-injector.ts·manuscript-uploader.tsx 정리	P0	S	Sonnet 5.5	단순 타입 오류와 죽은 코드 정리
4	업로드 계약 통일: 프론트 3곳을 use-manuscript-upload 단일 훅으로 합쳐 /api/upload/parse에 multipart 1회 호출, /manuscript hex 에코 제거, 20MB 상한, 파싱을 run_in_threadpool로, HWP는 zlib 해제 후 레코드 파싱 또는 지원 목록에서 제외	P0	M	Opus 5.5	명확한 계약 수정, 3~4개 파일
5	전체 집필 수정: use-write-all.ts를 fetch(POST) + ReadableStream SSE 파서로 교체(TOC·persona 전달, AbortController 중단), 결과를 manuscript 상태에 병합, 백엔드 TOC 길이 상한(예: 60)	P0	M	Opus 5.5	중간 규모 API 연동과 상태 반영
6	내보내기 복구: Settings에 weasyprint_server_url 추가, v1 export.py의 WeasyPrint + Noto CJK 렌더와 ePUB3 빌더 이식, 서버가 project_id로 원고를 조회해 HTML 생성, 모든 텍스트 html.escape/XML escape, 3,000자 절단 제거, 렌더는 스레드풀	P0	M	Opus 5.5	범위가 명확한 엔진 이식

#	작업	우선순위	예상규모	권장 모델	선정 이유
7	프로젝트 영속화: 위저드 진입 시 POST /api/projects로 ID 발급 후 /projects/ebook/[id]로 이동, use-ebook-step·use-upload-step이 ebook_step_state를 debounce PATCH, TEMP_PROJECT_ID='new'·DRAFT_PROJECT_ID 제거	P1	M	Opus 5.5	상태관리 정리, 2~5개 파일
8	페르소나·메타데이터 정합: 페르소나 60종을 JSON 단일 소스로 분리해 백엔드가 로드, geo-metadata가 DB의 제목·기획안·원고 앞부분을 프롬프트에 포함	P1	M	Opus 5.5	데이터 정합과 프롬프트 연동
9	v1 엔진 이식(표지·TTS): v1 cover.py 제공자 라우팅을 httpx.AsyncClient로 이식, TTS는 F5 /v1/speech 계약에 맞춰 결과를 Supabase Storage에 저장 후 서명 URL 반환, 오디오북 페이지가 URL을 받아 다운로드	P1	L	Opus 5.5	명확한 API 연동, 기존 코드 이식
10	블로킹 제거: supabase-py 호출을 run_in_threadpool 래퍼(또는 AsyncClient)로, Stripe 호출 asyncio.to_thread, 전체 집필을 세마포어 3 병렬화	P1	M	Opus 5.5	원인이 명확한 성능 수정
11	백엔드 오류 처리 정리: .single()을 maybe_single()로 바꾸고 404 처리, automation.py:35 raise, update_status 값 Enum 검증, 클라이언트로 str(e) 노출 축소 + logging.exception, datetime.utcnow 교체	P1	S	Sonnet 5.5	반복적인 단순 버그 수정
12	의존성·환경 정리: email-validator 추가, 미사용 numpy·soundfile·markdown 제거, pyproject.toml 전환, 루트 .env·.env.local 삭제와 키 재발급 안내, CLAUDE.md 레포 구조·실행 명령을 v2 실제 구조로 갱신	P2	S	Sonnet 5.5	설정 파일과 문서 정리
13	배포 설정 신설: api/Dockerfile(pango, cairo, fonts-noto-cjk), .dockerignore, Railway 또는 Render 설정, GitHub Actions CI(프론트 tsc/lint/build, 백엔드 pytest), Vercel env 목록 문서	P2	M	Opus 5.5	다수 설정 파일 작성과 검증
14	백엔드 테스트 설계: pytest + httpx AsyncClient로 무토큰 401, 위조 웹훅 400, 업로드 한도 413, 파서 단위 테스트(txt/md/docx), ePUB escape 테스트	P2	M	Opus 5.5	테스트 설계

6. 종합 평가

강점

1. 구조가 v1보다 확실히 좋아졌다. v1의 4,095줄 단일 위저드가 154줄 셸 + 단계별 컴포넌트로 분해되었고, 가장 큰 페이지도 491줄이다. 백엔드도 services/llm.py로 LLM 호출을 분리하고 비동기 SDK(AsyncAnthropic)를 사용한다.
2. 기획 문서와 도메인 데이터가 풍부하다. PRD/DESIGN/CLAUDE 문서가 10단계 파이프라인·플랜 한도·테마·컴포넌트 스펙을 구체적으로 정의하고, 60인 페르소나·6종 판형 Auto-Fit·7종 테마 데이터가 코드에 있다.
3. 보안 면에서 일부 위험 면적이 줄었다. v1의 SSRF(cover_url 다운로드), TTS 경로 조작 쓰기, dev-user 인증 우회, /debug-api-keys, 프론트 디렉터리의 서비스 롤 키가 사라졌고, 모든 테이블에 RLS가 켜졌다.

약점

1. 엔드투엔드로 동작하는 흐름이 없다. 로그인 세션 미생성, 토큰 미전송, 업로드·전체 집필·내보내기 계약 불일치, 빌드 실패가 겹쳐 v1보다 실사용 가능성이 낮다.

2. v1의 핵심 보안 부채(LLM 무인증, `user_plans` RLS 과다 허용, `'1234'` Pro 우회, 서버 플랜 미강제)가 그대로 넘어왔다.
3. 핵심 엔진이 스텁이다. 표지는 placehold.co, TTS는 결과 폐기, PDF는 한글 불가 fpdf2, 쪽번호는 추정치 메시지뿐이며 배포 설정이 전혀 없다.

v1(flora) 대비 비교

항목	v1 상태	v2 상태	판정
프론트 구조	4,095줄 단일 페이지, 1,422줄 복제	154줄 셸 + 단계 컴포넌트	개선
LLM 비동기	동기 SDK/requests	AsyncAnthropic	개선
LLM 제공자	Anthropic/OpenAI/Gemini/OpenRouter 폴백	Anthropic 단일	후퇴
LLM 인증	없음	없음	미해결
Stripe 웹훅	서명 미검증(권한 상승)	시크릿 오용으로 전부 400, 처리 로직 없음	위험 감소, 기능 없음
user_plans RLS	FOR ALL	FOR ALL	미해결
Pro 우회 `'1234'`	있음	있음	미해결
SSRF / TTS 경로 조작	있음	해당 기능 제거로 소멸	개선(기능 축소 대가)
인증 우회(dev-user)	있음	없음, 대신 빈 JWT secret 위험	부분 개선
waitlist	경로 중복 버그, in-memory	Supabase 테이블, 경로 정상	개선
프론트 토큰 전송	프로젝트 CRUD만 Bearer	전무	후퇴
PDF/ePUB	WeasyPrint/fpdf2/Typst, ePUB3	fpdf2(한글 불가), ePUB2, 본문 미전송	후퇴
표지/TTS	7종 제공자, F5 멀티보이스	플레이스홀더, 결과 폐기	후퇴
배포 설정	있으나 고장	없음	후퇴
`.single()` 500	있음	있음	미해결

flora / Flora-v2 / flora-publisher 처리 권고

- **Flora-v2: 본선으로 유지.** 문서 체계와 컴포넌트 구조가 세 저장소 중 가장 유지보수 가능하다. 단, 지금 상태로는 동작하지 않으므로 작업 1~7을 먼저 수행한다.
- **flora(v1): 엔진 이식 후 아카이브.** 작업 6·9에서 v1의 `api/routers/llm.py`(call_llm, OpenRouter 폴백), `cover.py` 제공자 7종, `export.py`(WeasyPrint/Typst/ePUB3), `frontend/src/lib/print-presets.ts`·`typst-presets.ts`·`html-templates.ts`, `f5-tts/api_server.py` 계약을 비동기화해 v2로 옮긴 뒤 `archive/flora-v1`로 이동한다. 이식 전에는 삭제하지 않는다.
- **flora-publisher: 아카이브(참고 자료).** README가 밝히듯 Vertex AI Studio에서 내보낸 "시장 조사" 데모(Express 프록시 455줄 + Vite 프론트, 소스 약 25개)로 FLORA 코드와 공유하는 부분이 없다. `frontend/docs/prd/*.md`의 사용자 흐름만 PRD에 반영하고 저장소는 아카이브한다. 같은 폴더의 zip(`flora-publisher-factory---market-research.zip`)도 중복이므로 함께 정리한다.

모델별 작업 비율

- Fable 5.1: 2건 (14%) — 인증 체계 복구, 플랜/결제 서버 강제
- Opus 5.5: 9건 (64%) — 업로드, 전체 집필, 내보내기, 영속화, 페르소나, 엔진 이식, 블로킹 제거, 배포, 테스트
- Sonnet 5.5: 3건 (22%) — 빌드 복구, 오류 처리, 의존성·문서 정리

권장 진행 순서

1. 1단계(동작과 보안 봉합, 1주): 작업 3 → 1 → 2. 빌드를 먼저 살리고, 세션·토큰·백엔드 인증·RLS·플랜 강제를 한 번에 맞춘다. 이 단계 전에는 공개 URL로 배포하지 않는다.
2. 2단계(핵심 파이프라인 완주, 1~2주): 작업 4 → 5 → 7 → 6 → 8. 업로드·집필·DB 영속화·내보내기를 연결해 "로그인 → 10단계 → PDF/ePUB 다운로드"가 실제로 완주되게 한다. 이때 v1 엔진을 이식한다.
3. 3단계(확장과 운영, 1~2주): 작업 9 → 10 → 11 → 12 → 13 → 14. 표지·TTS 이식, 성능, 오류 처리, 배포 파이프라인과 테스트를 갖춘 뒤 flora v1을 아카이브한다.

주 담당 모델: Opus 5.5

Flora-v2의 남은 문제 대부분은 설계 결함이 아니라 "프론트와 백엔드가 서로 다른 계약을 가정한" 배선 오류다. 업로드 프로토콜, SSE 메서드, 내보내기 바디, 프로젝트 ID 발급, 페르소나 키처럼 원인이 명확하고 2~5개 파일 범위에서 끝나는 수정이 작업의 64%를 차지하며, v1에 이미 검증된 구현이 있어 이식 방향도 정해져 있다. 따라서 주 담당은 Opus 5.5가 적절하다. 다만 인증 체계 복구(작업 1)와 플랜/결제·RLS(작업 2)는 실수 시 API 비용 유출과 무료 권한 상승으로 직결되므로 Fable 5.1에 맡기고, 빌드·오류 처리·문서 같은 기계적 정리는 Sonnet 5.5로 내려 비용을 줄인다.